

# Resumo de Estudo — Equivalência AP ↔ GLC (Parte 1)

## Conversão de GLC para AP • Seção 2.2

Já vimos que gramáticas livres-do-contexto e autômatos com pilha descrevem as mesmas linguagens (as LLCs). Agora vamos **provar** essa equivalência. Como todo teorema "se e somente se", a prova tem duas direções — e esta aula cobre a primeira: mostrar que, dada qualquer GLC, existe um AP equivalente.

## 1. O teorema da equivalência

### Teorema — AP e GLC têm o mesmo poder

Uma linguagem é livre-do-contexto **se e somente se** algum autômato com pilha a reconhece. A demonstração se divide em duas partes:

- **Direção 1 (esta aula):** se A é livre-do-contexto (gerada por uma GLC), então **algum AP a reconhece**.
- **Direção 2 (próxima aula):** se A é reconhecida por um AP, então **A é gerada por uma GLC**.

Nesta aula construímos, a partir de uma GLC G qualquer, um AP P que aceita exatamente  $L(G)$ . A pergunta central que P responde é: *existe uma sequência de substituições, usando as regras de G, que leva da variável inicial até a cadeia de entrada w?*

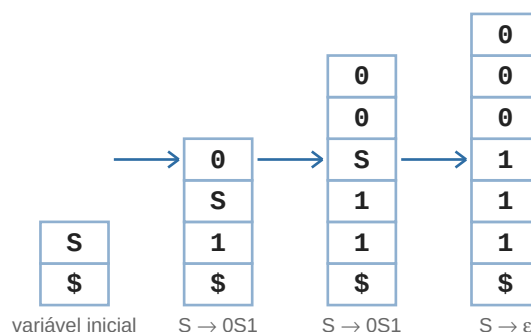
## 2. A ideia-chave: usar a pilha para as cadeias intermediárias

Lembre como uma GLC gera uma cadeia: partindo da variável inicial, vamos substituindo variáveis pelas regras até sobrar só terminais. O AP P vai **simular** esse processo, guardando na **pilha** a cadeia intermediária que está sendo derivada.

Vamos usar como exemplo  $L = \{ 0^n 1^n \}$ , com a GLC G:  $S \rightarrow 0S1 \mid \epsilon$ , derivando 000111:

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111$$

Uma primeira ideia ("hipotética") seria: (1) gerar toda a cadeia intermediária na pilha, aplicando as regras; (2) quando a pilha tiver só terminais, compará-los com a entrada, desempilhando quando o topo casa com a fita. Veja a fase 1 (empilhando a derivação de 000111):



*Fase 1 (hipotética): a pilha acumula a cadeia intermediária. A cabeça de leitura fica parada no início da fita durante esta fase.*

## O problema (e a correção)

A ideia hipotética tem um defeito fatal: **não dá para acessar/substituir símbolos que não estejam no topo da pilha**. Uma variável no meio da cadeia intermediária ficaria inacessível. A correção é **mesclar as duas fases** e trabalhar sempre no topo:

### Regra de operação de P (as duas situações no topo da pilha)

- **Se o topo é uma variável A:** desempilhe A e empilhe, em seu lugar, o lado direito de **uma** das regras de A (escolha não-determinística). Isso não consome entrada.
- **Se o topo é um terminal a:** compare com o símbolo atual da fita. Se forem iguais, desempilhe e avance a cabeça de leitura; se diferentes, este ramo do não-determinismo falha.

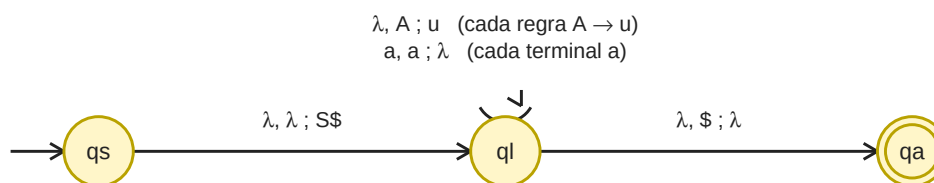
### 3. Construção do AP P (Lema 2.21)

#### Descrição informal (Sipser)

- 1. Empilhe o marcador \$ e a variável inicial S.
- 2. Repita, em cada ramo de computação possível:
- (a) Se o topo é uma variável A: escolha não-deterministicamente uma regra  $A \rightarrow u$  e empilhe u **em ordem reversa** (o primeiro símbolo de u fica no topo).
- (b) Se o topo é um terminal a: leia o próximo símbolo da entrada; se for igual a «a», continue; senão, rejeite neste ramo.
- (c) Se o topo é o marcador \$: entre no estado de aceitação (aceita se a entrada tiver sido totalmente lida).

#### Diagrama geral de P

Três estados bastam:  $q_{\text{start}}$  (inicializa),  $q_{\text{loop}}$  (faz todo o trabalho) e  $q_{\text{accept}}$ :



A aresta  $\lambda, \lambda; S\$$  empilha o inicial S sobre o sentinela \$. No laço de  $q_{\text{loop}}$  ficam TODAS as regras (uma transição  $\lambda, A; u$  por regra  $A \rightarrow u$ ) e o casamento de terminais ( $a, a; \lambda$ ). Quando o \$ aflora no topo, a transição  $\lambda, \$; \lambda$  leva à aceitação.

#### Detalhe técnico: empilhar uma cadeia (transições intermediárias)

A pilha só aceita um símbolo por transição, mas uma regra  $A \rightarrow u$  pode ter u com vários símbolos. A solução é quebrar isso em **transições intermediárias**, com estados auxiliares. E há um ponto crucial: **empilha-se em ordem reversa**, para que o primeiro símbolo de u termine no topo. Por exemplo, para a regra que substitui s por «xyz» ao ler «a»:

Uma transição «a,  $s \rightarrow xyz$ » vira uma cadeia de transições:

$q \xrightarrow{(a, s \rightarrow z)} q_1 \xrightarrow{(\lambda, \lambda \rightarrow y)} q_2 \xrightarrow{(\lambda, \lambda \rightarrow x)} r$

Resultado na pilha (topo em cima): x / y / z (x no topo)

Empilhamos z primeiro (vai para o fundo), depois y, e por fim x (fica no topo). Assim, ao lermos a cadeia da esquerda para a direita, o topo apresenta os símbolos na ordem correta x, y, z.

## 4. Exemplo completo de conversão

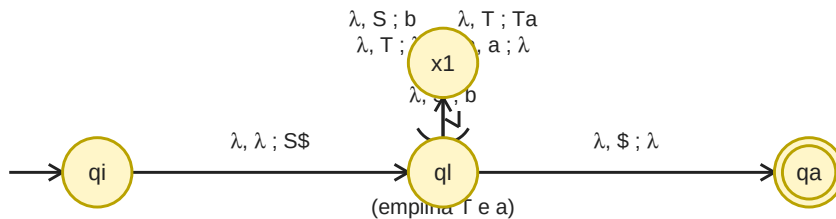
Convertendo a GLC abaixo em um AP equivalente:

**GLC**

$$S \rightarrow aTb \mid b$$

$$T \rightarrow Ta \mid \varepsilon$$

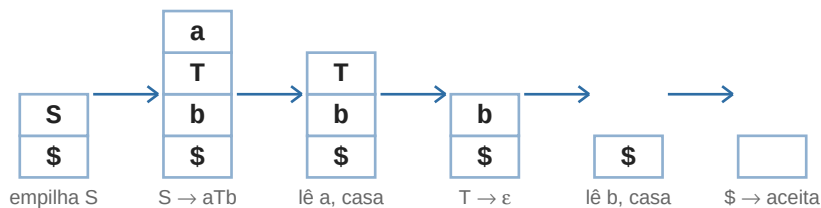
O AP tem  $q_{\text{início}}$ ,  $q_{\text{laço}}$ ,  $q_{\text{aceita}}$  e alguns estados intermediários para empilhar as cadeias de mais de um símbolo (como «aTb», empilhada como b, depois T, depois a):



As transições do laço de  $q_{\text{laço}}$  tratam as regras diretas ( $S \rightarrow b$ ,  $T \rightarrow \lambda$ ) e o casamento de terminais ( $a, a; \lambda$  e  $b, b; \lambda$ ). Para a regra  $S \rightarrow aTb$ , que tem três símbolos, empilha-se b, depois T, depois a (ordem reversa) usando um estado intermediário; o mesmo vale para  $T \rightarrow Ta$ . Por clareza, a simulação abaixo mostra o efeito na pilha, que é o que realmente importa para o entendimento.

### Simulando P sobre a entrada «ab»

A cadeia «ab» é gerada por  $S \Rightarrow aTb \Rightarrow ab$  (usando  $T \rightarrow \varepsilon$ ). Veja como a pilha de P reproduz essa derivação (topo em cima):



Cada passo ou expande a variável do topo (por uma regra) ou casa um terminal do topo com a fita. Ao aflorar o \$ com a entrada esgotada, P aceita «ab». ✓

## Exercícios resolvidos

### Exercício 1 — Converta $G: S \rightarrow 0S1 \mid \varepsilon$ em um AP

Aplicamos a construção do Lema 2.21. Três estados principais mais um intermediário para empilhar «0S1» (três símbolos):

```
q_início --( $\lambda, \lambda \rightarrow S\$$ )--> q_laço
```

No laço q\_laço:

$\lambda, S \rightarrow 1$  (início da regra  $S \rightarrow 0S1$ : empilha 1)  
seguido de  $\lambda, \lambda \rightarrow S$  e  $\lambda, \lambda \rightarrow 0$  (ordem reversa: topo fica 0)

$\lambda, S \rightarrow \lambda$  (regra  $S \rightarrow \varepsilon$ )

$0, 0 \rightarrow \lambda$  (casa terminal 0)

$1, 1 \rightarrow \lambda$  (casa terminal 1)

```
q_laço --( $\lambda, \$ \rightarrow \lambda$ )--> q_aceita
```

A regra  $S \rightarrow 0S1$  é empilhada como 1 (fundo), S (meio), 0 (topo), de modo que a leitura processe primeiro o 0, depois expanda S, e por fim case o 1. ✓

### Exercício 2 — Por que empilhar em ordem reversa?

Porque a pilha é LIFO: o último símbolo empilhado é o primeiro a ser lido (fica no topo). Se a regra é  $A \rightarrow xyz$  e queremos que a derivação continue processando x, depois y, depois z (a ordem natural de leitura da cadeia), precisamos que x esteja no topo. Logo empilhamos na ordem inversa: z primeiro (vai ao fundo), y, e x por último (fica no topo). Se empilhássemos na ordem direta, leríamos a cadeia invertida (zyx), o que estaria errado. ✓

**Exercício 3 — Simule o AP de  $S \rightarrow 0S1 \mid \epsilon$  sobre a entrada «0011»**

A derivação em G é  $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$ . A pilha de P (topo em cima) acompanha:

| pilha (topo→fundo) | ação                                                 |
|--------------------|------------------------------------------------------|
| S \$               | empilha inicial                                      |
| 0 S 1 \$           | $S \rightarrow 0S1$                                  |
| S 1 \$             | lê 0, casa com topo                                  |
| 0 S 1 1 \$         | $S \rightarrow 0S1$                                  |
| S 1 1 \$           | lê 0, casa com topo                                  |
| 1 1 \$             | $S \rightarrow \epsilon$                             |
| 1 \$               | lê 1, casa com topo                                  |
| \$                 | lê 1, casa com topo                                  |
| (vazia)            | $\lambda, \$ \rightarrow \lambda \rightarrow$ ACEITA |

A entrada «0011» é consumida exatamente quando o \$ aflora  $\rightarrow$  aceita. ✓